

Multi-Agent GitHub Code Review & Autonomous Auto-Fix System

Complete Architecture Design Document

High-Level Design • Low-Level Design • Technology Stack • Implementation Guide

Version	1.0.0
Date	June 2026
Author	Skpandey15
Stack	Java 17 + Spring Boot 3.3.4 + LangChain4j + Temporal.io + GPT-4o
Repository	github.com/Skpandey15/-multi-agent-github-review

Confidential — Internal Engineering Document

Table of Contents

1.	Executive Summary & Problem Statement	3
2.	Why This Product is Required	3
3.	System Overview	4
4.	High-Level Design (HLD)	5
4.1	Component Architecture	5
4.2	Data Flow Diagram	6
4.3	Deployment Architecture	6
5.	6-Agent Pipeline — Detailed Design	7
5.1	Agent-1: Repository Reader	7
5.2	Agent-2: LLM Analyser	7
5.3	Agent-3: Adversarial Critic	8
5.4	Agent-4: Code Fixer	8
5.5	Agent-5: PR Raiser	9
5.6	Agent-6: Notifier	9
6.	Low-Level Design (LLD)	10
6.1	Database Schema	10
6.2	Temporal Workflow Design	11
6.3	API Specification	12
6.4	GraphQL Schema	13
6.5	Error Handling Strategy	13
6.6	MDC Distributed Tracing	14
7.	Technology Stack	14
8.	Security Design	15
9.	Scalability & Resilience	16
10.	Implementation Guide	17
11.	End-to-End Flow Walkthrough	18
12.	Key Design Decisions & Trade-offs	19

1. Executive Summary & Problem Statement

The **Multi-Agent GitHub Code Review & Auto-Fix System** is a production-grade, AI-powered pipeline that autonomously reviews any GitHub repository, identifies security vulnerabilities and code quality issues, applies targeted fixes, raises a human-reviewable Pull Request, and sends email notifications — all without any human involvement in the analysis phase.

The system is built on a **6-agent LLM pipeline** orchestrated by Temporal.io for durable, fault-tolerant execution. An adversarial Critic agent eliminates false positives before any code is touched. The human remains in the loop at the PR review stage — the system **never auto-merges**.

2. Why This Product is Required

Pain Point	Impact	Our Solution
Manual code review is slow (days per PR)	Delayed releases, bottleneck on senior devs	Automated analysis in < 5 min per repository
Security vulnerabilities slip into production	Data breaches, compliance failures	GPT-4o detects OWASP Top-10, SQL injection, XSS, hardcoded secrets
Inconsistent review quality across reviewers	Technical debt, unpredictable quality	Deterministic 6-agent pipeline with adversarial validation
False positives waste developer time	Review fatigue, ignored warnings	Adversarial Critic eliminates false positives before fixing
No traceability of AI suggestions	Lack of audit trail	PostgreSQL persistence of every finding + Critic reasoning
On-call devs miss critical issues at night	Incidents go undetected	Nightly scheduler + email alerts on completion

Key business outcomes delivered:

- 90%+ reduction in time-to-review for standard code quality checks
- Consistent security scanning on every repository trigger — no human scheduling required
- Full audit trail: every issue found, Critic verdict, fix applied, and PR raised are stored in PostgreSQL
- Zero false-positive auto-fixes: confidence threshold (≥75%) + severity filter (HIGH/MEDIUM) + Critic CONFIRMED
- Human always reviews the final PR — AI assists, humans decide

3. System Overview

The system exposes three entry points: a **REST trigger API**, a **GitHub webhook** (HMAC-verified), and a **nightly cron scheduler**. All three paths converge on a single Temporal workflow that executes the 6-agent pipeline sequentially with full retry/backoff guarantees.

6-Agent Pipeline



Reader → Analyser → Critic → Fixer → PR Raiser → Notifier

Agent	Role	LLM?	Output
Reader	Clones repo, maps files to CodeMap object	No	CodeMap (files + metadata)
Analyser	Finds security & quality issues	GPT-4o	List
Critic	Adversarially refutes each issue	GPT-4o	List
Fixer	Generates patch for confirmed issues	GPT-4o	List
PR Raiser	Creates branch, commits, opens PR	No (GitHub API)	PR URL
Notifier	Sends email, persists to PostgreSQL	No	Stored findings + email

4. High-Level Design (HLD)

4.1 Component Architecture

The system is composed of five logical layers. Each layer has a single responsibility and communicates through well-defined interfaces.

Layer	Components	Responsibility
Entry Layer	TriggerController (REST) WebhookController (GitHub) NightlyReviewScheduler (Cron)	Accept review requests from all sources, validate HMAC signatures, enqueue to Temporal
Orchestration Layer	CodeReviewWorkflowImpl (Temporal) TemporalConfig	Durable workflow execution, activity scheduling, retry/backoff, saga-style error compensation
Agent Layer	GitHubActivitiesImpl LLMActivitiesImpl NotificationActivitiesImpl	Execute the 6-agent pipeline steps: read → analyse → criticise → fix → PR → notify
Intelligence Layer	AnalyserAgent (LangChain4j) CriticAgent (LangChain4j) FixerAgent (LangChain4j)	LLM-powered AI services using GPT-4o via AiServices.builder() interface pattern
Persistence Layer	PostgreSQL 16 + Flyway PipelineRunRepository FindingRepository	Store every run, finding, fix, Critic verdict with full audit trail
Observability Layer	Temporal UI (:8080) GraphiQL (:8081/graphiql) MDC + correlationId logging	Workflow visibility, query interface, distributed trace correlation

4.2 Data Flow Diagram

Below is the complete data flow from trigger to PR creation. Numbers correspond to the sequence of operations within the Temporal workflow.

Step	From	To	Data / Action
1	Client / GitHub / Cron	TriggerController / WebhookController	repoUrl, branch, triggerType
2	TriggerController	PipelineService	Create PipelineRun (PENDING) in PostgreSQL
3	PipelineService	Temporal Workflow	Start CodeReviewWorkflow with runId + repoUrl
4	Temporal	GitHubActivitiesImpl.readRepository()	Clone repo, walk file tree
5	readRepository()	LangChain4j AnalyserAgent	CodeMap (files, language, framework)
6	AnalyserAgent → GPT-4o	LLMActivitiesImpl	54 IssueFound objects (example run)
7	Analyser output	CriticAgent → GPT-4o	Each issue sent for adversarial review
8	CriticAgent	Fixer filter	37 CONFIRMED, 17 FALSE_POSITIVE eliminated
9	Confirmed issues (HIGH/MED + conf≥75%)	FixerAgent → GPT-4o	15 FixResult patches generated

Step	From	To	Data / Action
10	FixerAgent	GitHubActivitiesImpl.createPullRequest()	Commit patches to new branch
11	GitHub API	Temporal	PR URL returned
12	Temporal	NotificationActivitiesImpl	Save all findings to DB, send email
13	PostgreSQL	GraphQL / REST API	Queryable via /graphql or /api/runs/{id}

4.3 Deployment Architecture (Docker Compose)

All services run in a single Docker Compose network. The application container is the only service with outbound internet access (GitHub API + OpenAI API).

Container	Image	Port	Role	Depends On
postgres	postgres:16-alpine	5432	Primary datastore	—
temporal	temporalio/auto-setup:1.25.1	7233 (gRPC)	Workflow engine + DB backend	postgres
temporal-ui	temporalio/ui:2.26.2	8080 (HTTP)	Workflow visibility dashboard	temporal
app	Custom (eclipse-temurin:17-jre)	8081 (HTTP)	Spring Boot application + all agents	temporal, postgres

5. 6-Agent Pipeline — Detailed Design

5.1 Agent-1: Repository Reader

Implemented as a **Temporal Activity** (*GitHubActivitiesImpl.readRepository()*). Uses the **kohsuke/github-api** library to authenticate and clone the target repository. Walks the file tree recursively, filtering to supported source file extensions.

Attribute	Detail
Input	repoUrl (e.g. https://github.com/owner/repo.git), branch name
Process	Extract owner/repo (strip .git suffix), call GitHub API, walk GHTree recursively, filter by extension (.java .py .js .ts .go .rb .php .cs .cpp .kt .rs .swift)
Output	CodeMap { repoUrl, branch, detectedLanguage, detectedFramework, List, totalFiles }
File size limit	Files > 100 KB skipped to avoid exceeding LLM context window
Max files	200 files per run (configurable) to control cost and latency
Error handling	GitHub 404 → GitHubApiException. Retry: maxAttempts=3, backoff 2s→4s→8s
Activity timeout	scheduleToCloseTimeout = 10 min, startToCloseTimeout = 8 min

5.2 Agent-2: LLM Analyser

Powered by **GPT-4o** via LangChain4j *AiServices.builder()*. The *AnalyserAgent* interface defines a single method that accepts the serialized CodeMap and returns a structured JSON list of issues.

Attribute	Detail
Interface method	List analyseCode(String codeMapJson)
System prompt scope	OWASP Top-10, SQL injection, XSS, hardcoded secrets, JWT misconfiguration, null pointer risks, resource leaks, missing auth checks, deprecated APIs
Output structure	IssueFound { filePath, lineNumber, issueType, severity (HIGH/MEDIUM/LOW), confidence (0-100), description, originalCode, suggestedFix }
LLM model	gpt-4o (configured via OPENAI_API_KEY env var)
Temperature	0.1 — low temperature for deterministic security analysis
Max tokens	4096 output tokens per call
Typical findings	54 issues on auth-service-jwt (JWT hardcoded secret, missing token expiry, etc.)

5.3 Agent-3: Adversarial Critic

The Critic is a **second independent LLM call** that attempts to **REFUTE** each finding from the Analyser. This is the key differentiator — it eliminates false positives before any code is modified. The Critic is prompted to be skeptical by default.

Attribute	Detail
Interface method	CriticVerdict reviewFinding(String issueJson, String relevantCode)

Attribute	Detail
Adversarial prompt	"You are a strict senior engineer. Try to REFUTE this finding. Only CONFIRM if the evidence is clear and unambiguous."
Output structure	CriticVerdict { verdict (CONFIRMED/FALSE_POSITIVE/NEEDS_MORE_CONTEXT), reasoning, adjustedSeverity, adjustedConfidence }
False positive rate	17 out of 54 eliminated (31%) on auth-service-jwt test run
Confirmed issues	37 issues passed Critic review for potential fixing
Each issue	One independent LLM call — no cross-contamination between findings

5.4 Agent-4: Code Fixer

The Fixer applies a **strict eligibility filter** before calling GPT-4o. Only issues that pass all three criteria are auto-fixed. This prevents AI from touching code it is not confident about.

Eligibility Filter (all three must pass):

Criterion	Value	Rationale
Critic verdict	CONFIRMED	False positives already eliminated by Agent-3
Severity	HIGH or MEDIUM	LOW issues not worth automated changes — human judgement preferred
Confidence score	≥ 75%	High-confidence fixes only to avoid introducing new bugs

Attribute	Detail
Interface method	FixResult generateFix(String issueJson, String originalCode)
Output structure	FixResult { filePath, fixedCode, commitMessage, explanation }
Fix scope	Single-file patches only — no cross-file refactoring (safety constraint)
Commit message style	Conventional Commits: fix(security):
Example fix	Replaced hardcoded JWT secret with environment variable reference Added @NotNull validation on token parameter
Fixes applied	15 out of 37 confirmed issues on auth-service-jwt test run

5.5 Agent-5: PR Raiser

Creates a new git branch, commits all patches atomically, and opens a GitHub Pull Request using the kohsuke/github-api library. The human engineer reviews and merges — **the system never auto-merges**.

Attribute	Detail
Branch naming	code-review/ai-fixes-{runId} — unique per run, no conflicts
Commit strategy	One commit per file changed — clean git history, easy partial revert
PR title	"[AI Review] {N} security and quality fixes — {repoName}"
PR body	Markdown table listing all fixes with file path, issue type, severity
GitHub auth	Fine-grained PAT with Contents: Read & Write + Pull Requests: Read & Write
Base branch	main (configurable via trigger request)
Human gate	PR requires human approval to merge — system cannot auto-merge

5.6 Agent-6: Notifier

Handles two responsibilities: **persisting all findings** to PostgreSQL (including Critic reasoning for audit) and **sending email notifications** via Gmail SMTP. Email delivery is **non-fatal** — if Gmail auth fails, the workflow succeeds anyway.

Attribute	Detail
Email 1 — Analysis Complete	Sent after Agent-3. Contains: run ID, issues found, false positives eliminated
Email 2 — PR Raised	Sent after Agent-5. Contains: PR URL, issues fixed, issues skipped, direct link to PR
SMTP config	Gmail SMTP, port 587, STARTTLS, App Password authentication
Non-fatal design	try/catch wraps all email calls — log.warn() on failure, workflow continues
DB persistence	Every finding saved: filePath, severity, status, confidence, fixedCode, criticReasoning
Status updates	PipelineRun status updated: PENDING → RUNNING → COMPLETED/FAILED

6. Low-Level Design (LLD)

6.1 Database Schema

Two application tables managed by Flyway migrations (V1__init.sql). PostgreSQL 16 on port 5432, database: *github_review*.

Table: pipeline_runs

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Unique run identifier (auto-generated)
repo_url	VARCHAR(500)	NOT NULL	Full GitHub repository URL
branch	VARCHAR(255)	NOT NULL, DEFAULT 'main'	Target branch analysed
status	VARCHAR(50)	NOT NULL	PENDING RUNNING COMPLETED FAILED
trigger_type	VARCHAR(50)	NOT NULL	MANUAL WEBHOOK SCHEDULED
total_issues_found	INTEGER	DEFAULT 0	Count from Analyser output
total_issues_fixed	INTEGER	DEFAULT 0	Count of applied patches
total_false_positives	INTEGER	DEFAULT 0	Count eliminated by Critic
pr_url	VARCHAR(500)		GitHub PR URL (null until PR raised)
pr_branch	VARCHAR(255)		Name of the AI-created branch
error_message	TEXT		Populated only on FAILED runs
started_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Workflow start time
completed_at	TIMESTAMPTZ		Null until terminal status reached

Table: findings

Column	Type	Constraints	Description
id	UUID	PK, NOT NULL	Finding identifier
run_id	UUID	FK → pipeline_runs.id	Parent run reference
file_path	VARCHAR(500)	NOT NULL	Relative file path in repo
line_number	INTEGER		Line number of the issue
issue_type	VARCHAR(100)	NOT NULL	E.g. SQL_INJECTION, HARDCODED_SECRET
severity	VARCHAR(20)	NOT NULL	HIGH MEDIUM LOW

Column	Type	Constraints	Description
confidence	INTEGER	CHECK 0-100	LLM confidence score
status	VARCHAR(50)	NOT NULL	FIXED SKIPPED FALSE_POSITIVE
description	TEXT	NOT NULL	Human-readable issue description
original_code	TEXT		Vulnerable code snippet
fixed_code	TEXT		AI-generated replacement code
commit_message	VARCHAR(500)		Git commit message for the fix
critic_reasoning	TEXT		Critic's explanation of verdict
created_at	TIMESTAMPTZ	NOT NULL, DEFAULT NOW()	Finding creation time

6.2 Temporal Workflow Design

The *CodeReviewWorkflowImpl* is a Temporal workflow that orchestrates all 6 activities sequentially. Temporal provides automatic retry, state persistence, and resumability — if the worker crashes mid-workflow, it resumes from the last completed activity.

Activity	scheduleToClose	startToClose	maxAttempts	Backoff
readRepository	10 min	8 min	3	2s, 4s, 8s
analyseCode	15 min	12 min	2	5s, 10s
reviewFinding (per issue)	5 min	4 min	3	2s, 4s, 8s
generateFix (per issue)	5 min	4 min	3	2s, 4s, 8s
createPullRequest	10 min	8 min	3	5s, 10s, 20s
sendAnalysisCompleteEmail	2 min	1 min	1	N/A (non-fatal)
sendPrRaisedEmail	2 min	1 min	1	N/A (non-fatal)
updateRunStatus	1 min	30s	5	1s, 2s, 4s, 8s, 16s
saveFinding	2 min	1 min	5	1s, 2s, 4s, 8s, 16s

Workflow State Machine:

State	Trigger	Next State
PENDING	Workflow started	RUNNING
RUNNING	readRepository() completes	Still RUNNING (Analyser starts)
RUNNING	All activities complete, PR raised	COMPLETED
RUNNING	Any non-retryable exception	FAILED (error saved to DB)
FAILED	Manual re-trigger with same repoUrl	New PENDING run created

6.3 REST API Specification

Method	Path	Auth	Request Body	Response
POST	/api/review/trigger	None (internal)	{ "repoUrl": "...", "branch": "main" }	{ "runId": "uuid", "status": "PENDING", "message": "..." }
GET	/api/review/status/{runId}	None	—	{ "runId": "...", "status": "COMPLETED", "prUrl": "...", ... }
POST	/api/webhook/github	X-Hub-Signature-256 (HMAC-SHA256)	GitHub push event JSON	{ "runId": "...", "message": "Review triggered" }
GET	/actuator/health	None	—	{ "status": "UP" }
GET	/graphql	None	—	GraphiQL UI (HTML)
POST	/graphql	None	GraphQL query JSON	GraphQL response JSON

6.4 GraphQL Schema

The GraphQL API (Spring for GraphQL) exposes two queries for querying pipeline results:

```
type Query {  
  allRuns: [PipelineRun!]! # All pipeline runs (newest first)  
  pipelineRun(id: ID!): PipelineRun # Single run by UUID  
}  
  
type PipelineRun {  
  id: ID! repoUrl: String! branch: String!  
  status: String! triggerType: String!  
  totalIssuesFound: Int totalIssuesFixed: Int totalFalsePositives: Int  
  prUrl: String prBranch: String errorMessage: String  
  startedAt: String completedAt: String  
  findings: [Finding!]!  
}  
  
type Finding {  
  id: ID! filePath: String! lineNumber: Int  
  issueType: String! severity: String! confidence: Int  
  status: String! description: String!  
  originalCode: String fixedCode: String commitMessage: String  
  criticReasoning: String createdAt: String  
}
```

6.5 Error Handling Strategy

The system uses a layered error handling approach: Temporal retries for transient failures, RFC 7807 ProblemDetail for REST API errors, and non-fatal try/catch for non-critical paths.

Error Type	Handler	Behaviour
GitHub API 404 (repo not found)	GitHubApiException → Temporal retry x3	Retry with backoff; FAILED if all retries exhausted
GitHub 403 (token insufficient)	GitHubApiException → No retry	Immediate FAILED — no point retrying auth errors
OpenAI API timeout	Temporal retry x2	Retry with 5s backoff; FAILED if both fail
OpenAI API rate limit (429)	Temporal retry x3	Retry with 10s backoff (respects API limits)
Email delivery failure	Non-fatal catch	log.warn() — workflow continues as COMPLETED
PostgreSQL write failure	Temporal retry x5	Critical — retry with backoff; FAILED if all fail
REST API validation error	GlobalExceptionHandler → RFC 7807 ProblemDetail	HTTP 400 with structured error detail
Internal server error	GlobalExceptionHandler → RFC 7807 ProblemDetail	HTTP 500 with correlationId for tracing
Webhook HMAC mismatch	WebhookController → HTTP 401	Request rejected immediately — no workflow started

6.6 MDC Distributed Tracing

Every request gets a **correlationId** (UUID) injected by *CorrelationIdFilter* at the HTTP boundary. The *MdcContextPropagator* carries this ID across Temporal activity thread boundaries so a single pipeline run can be traced across all log lines.

Component	Role
CorrelationIdFilter (OncePerRequestFilter)	Reads X-Correlation-Id header or generates new UUID; puts in MDC
MdcContextPropagator (ContextPropagator)	Serialises MDC map into Temporal workflow headers on dispatch
MdcContextPropagator.setUp()	Deserialises headers back into MDC on each activity worker thread
logback-spring.xml	Appends %X{correlationId} and %X{runId} to every log line
Log correlation query	grep 'correlationId=abc-123' app.log — shows full trace for one request

7. Technology Stack

Category	Technology	Version	Purpose
Runtime	Java (Microsoft OpenJDK)	17.0.19	Primary language runtime
Framework	Spring Boot	3.3.4	Application framework, DI, REST, GraphQL, mail
Build	Gradle (Groovy DSL)	8.x	Build system, dependency management
LLM SDK	LangChain4j	0.35.0	AiServices.builder() for GPT-4o integration
LLM Model	OpenAI GPT-4o	Latest	Security analysis, critic review, code fixing
Workflow	Temporal.io SDK	1.24.2	Durable activity orchestration + retry
Workflow Boot	temporal-spring-boot-autoconfigure	1.24.2	Auto-configures Temporal worker from YAML
GitHub API	kohsuke/github-api	1.321	Repo clone, branch create, PR creation
Database	PostgreSQL	16	Persistent storage for runs and findings
Migrations	Flyway	10.x (via Spring Boot)	Schema version control
ORM	Spring Data JPA + Hibernate	3.3.4 / 6.x	Repository pattern for DB access
API	Spring MVC (REST) + Spring GraphQL	3.3.4	REST endpoints + GraphQL query layer
GraphQL UI	GraphiQL	3.7.2 (self-hosted)	Browser-based query explorer (no CDN)
Email	Spring Mail + JavaMail	3.3.4	SMTP email via Gmail App Password
Containers	Docker + Docker Compose	Latest	Container runtime and orchestration
Base Image	eclipse-temurin:17-jre-alpine	Latest	Minimal JRE container (no build tools)
Logging	Logback + SLF4J + MDC	Via Spring Boot	Structured JSON logging with trace correlation
Validation	Jakarta Bean Validation	3.x	@Valid, @NotBlank on all DTOs
Error format	RFC 7807 ProblemDetail	Spring 6 built-in	Machine-readable REST error responses
Security	HMAC-SHA256	JDK built-in	GitHub webhook signature verification

8. Security Design

Security Control	Implementation	Threat Mitigated
Secrets management	All secrets via environment variables (.env file, never committed)	Credential exposure in source code or git history
GitHub webhook auth	HMAC-SHA256 of payload with shared secret, constant-time comparison via MessageDigest.isEqual()	Forged webhook requests triggering unauthorised reviews
GitHub token scope	Fine-grained PAT: Contents R/W + Pull Requests R/W No admin, no delete, no org-level permissions	Token abuse if compromised — minimal blast radius
No auto-merge	System raises PR, never merges. GitHub branch protection enforces human review	Malicious or incorrect AI code reaching production
LLM output validation	Structured output via LangChain4j typed interfaces. Jackson deserialization validates schema on receipt	Prompt injection in repo code affecting system behaviour
Single-file patches only	Fixer is constrained to one file per fix. No cross-file refactoring allowed	AI introducing systemic changes across the codebase
Confidence threshold	Only fixes with confidence $\geq 75\%$ applied	Low-confidence AI fixes introducing bugs
Docker network isolation	All containers on internal bridge network. Only app container reaches internet	Lateral movement if a container is compromised
Temporal mTLS (optional)	temporal-spring-boot-autoconfigure supports TLS config (TEMPORAL_TLS_* env vars)	Plaintext workflow data on gRPC channel
SQL injection prevention	Spring Data JPA parameterized queries throughout. No raw SQL string concatenation	DB compromise via crafted input in trigger API

9. Scalability & Resilience

9.1 Horizontal Scaling

Dimension	Current (Single Node)	Scaled Approach
Application workers	1 Docker container, 1 Temporal worker	Add more app containers — all share same task queue. Temporal distributes activities automatically.
Temporal server	Single node (auto-setup image)	Temporal Cloud or multi-node cluster for production. No code change required — just config.
Database	Single PostgreSQL instance	PostgreSQL read replicas for GraphQL queries. Connection pooling via HikariCP (Spring default).
LLM throughput	Sequential per-finding calls	Parallel activity dispatch for Critic/Fixer calls (Temporal fan-out pattern — workflow code change).
File limit	200 files / 100 KB per file	Increase limits + chunk large files across multiple LLM calls with sliding context window.

9.2 Resilience Features

- **Temporal durable execution:** if the worker crashes mid-workflow, Temporal replays from the last completed activity. No work is lost.
- **Activity retry policies:** every activity has configurable maxAttempts with exponential backoff. Transient API failures auto-recover.
- **Non-fatal email:** Gmail auth failure does not fail the workflow — the PR is still raised and findings still saved.
- **Idempotent DB writes:** PipelineRun created before workflow starts, updates are upserts — safe to replay.
- **Health endpoint:** /actuator/health exposes Spring Boot health. Docker Compose uses it as a healthcheck.
- **Correlation tracing:** every log line carries correlationId and runId — post-mortem analysis of failed runs is trivial.

10. Implementation Guide

10.1 Prerequisites

Requirement	Version	Notes
Java (Microsoft OpenJDK)	17+	Required for Spring Boot 3.x
Docker Desktop / Rancher Desktop	Latest	Docker Compose v2 required
GitHub Account	Any	Fine-grained PAT with Contents + Pull Requests R/W
OpenAI API Key	GPT-4o access	Paid tier required — GPT-4o not on free tier
Gmail Account	Any	App Password required (not account password)

10.2 Step-by-Step Setup

Step 1: Clone the repository

```
git clone https://github.com/Skpandey15/-multi-agent-github-review.git
cd -multi-agent-github-review
```

Step 2: Copy and fill .env

```
cp .env.example .env
# Edit .env with your real values:
# GITHUB_TOKEN, OPENAI_API_KEY, GMAIL_USERNAME,
# GMAIL_APP_PASSWORD, APP_EMAIL_TO
```

Step 3: Build the application JAR

```
./gradlew clean build -x test
```

Step 4: Start all containers

```
docker compose up -d --build
# Starts: postgres, temporal, temporal-ui, app
```

Step 5: Verify all containers are healthy

```
docker compose ps
# All 4 containers should show 'healthy' or 'running'
```

Step 6: Trigger a code review

```
curl -X POST http://localhost:8081/api/review/trigger \
-H "Content-Type: application/json" \
-d '{"repoUrl":"https://github.com/owner/repo.git","branch":"main"}'
```

Step 7: Monitor workflow progress

```
# Temporal UI: http://localhost:8080
# App logs: docker compose logs -f app
```

Step 8: Query results via GraphQL

```
# Open: http://localhost:8081/graphql
# Run: query AllRuns { allRuns { id status prUrl totalIssuesFixed } }
```

Step 9: Query results via PostgreSQL

```
psql -h localhost -U postgres -d github_review  
SELECT id, status, total_issues_found, pr_url FROM pipeline_runs ORDER BY started_at  
DESC;
```

11. End-to-End Flow Walkthrough

This section traces the exact execution path of the proven test run against *auth-service-jwt* repository — 54 issues found, 17 false positives eliminated, 15 fixes applied, PR raised.

#	Actor	Action	Duration	Result
1	Developer	POST /api/review/trigger with auth-service-jwt URL	~10ms	runId returned, PENDING status
2	PipelineService	Create PipelineRun in PostgreSQL, start Temporal workflow	~50ms	Workflow ID created
3	Temporal	Assign workflow to code-review-worker on code-review-queue	~100ms	Activity readRepository() scheduled
4	Agent-1 (Reader)	Authenticate GitHub API, walk 23 Java files, build CodeMap	~8s	CodeMap with 23 CodeFile objects
5	Agent-2 (Analyser)	Send CodeMap JSON to GPT-4o for security analysis	~45s	54 IssueFound objects
6	Temporal	Update run: totalIssuesFound=54, status stays RUNNING	~10ms	DB updated
7	Agent-3 (Critic)	Send each of 54 issues individually to GPT-4o for adversarial review	~3 min	37 CONFIRMED, 17 FALSE_POSITIVE
8	Temporal	Email 1 sent: '54 issues found, 17 false positives eliminated'	~2s	Gmail delivered (or non-fatal warn)
9	Fixer filter	Apply eligibility: CONFIRMED + HIGH/MED + confidence≥75%	~1ms	15 issues pass filter
10	Agent-4 (Fixer)	Send each of 15 issues to GPT-4o for patch generation	~90s	15 FixResult patches
11	Agent-5 (PR Raiser)	Create branch, commit 15 patches, open PR on GitHub	~15s	PR URL: /pull/5
12	Agent-6 (Notifier)	Save all 54 findings to PostgreSQL, send PR email	~5s	DB written, email sent
13	Temporal	Mark workflow COMPLETED	~10ms	Run status = COMPLETED
14	Developer	Receive email with PR link, review AI changes, merge or reject	Human time	Human decision — AI done

Observed Metrics (auth-service-jwt test run):

Metric	Value
Total files analysed	23 Java files
Total issues found by Analyser	54
False positives eliminated by Critic	17 (31%)
Issues confirmed by Critic	37

Metric	Value
Issues passing fixer eligibility filter	15
Patches applied and committed	15
GitHub PR URL	https://github.com/Skpandey15/auth-service-jwt/pull/5
Total pipeline duration	~6 minutes
Email notifications sent	2 (Analysis Complete + PR Raised)

12. Key Design Decisions & Trade-offs

Why Temporal.io instead of Spring @Async or Kafka?

Rationale: Temporal provides durable execution — if the JVM crashes mid-pipeline, the workflow resumes from the last completed step. Spring @Async loses all state on crash. Kafka requires manual checkpoint management and is harder to debug. Temporal's UI also gives instant visibility into workflow progress without additional tooling.

Trade-off: Higher operational complexity (separate Temporal server). Mitigated by temporal auto-setup image for dev.

Why LangChain4j AiServices.builder() instead of raw OpenAI API calls?

Rationale: AiServices.builder() generates type-safe Java proxies at runtime. The interface method signature defines input/output schema — Jackson handles serialization automatically. This is far less error-prone than parsing raw JSON strings. The @SystemMessage annotation co-locates the prompt with the interface definition.

Trade-off: LangChain4j 0.35.0 does not yet support streaming responses. Full response must arrive before the activity returns — no incremental progress for long analyses.

Why a separate Critic agent instead of asking the Analyser to self-validate?

Rationale: Self-validation by the same model with the same system prompt produces confirmation bias — the LLM tends to confirm its own findings. An independent Critic call with an adversarial system prompt ('try to REFUTE this') produces genuinely critical review. The 31% false-positive elimination rate in the test run validates this design choice.

Trade-off: Doubles the LLM API cost for the review phase. Mitigated by the value of high-precision fixes.

Why single-file patches only?

Rationale: Multi-file refactoring risks unintended side effects across the codebase. AI is most reliable when the fix scope is narrow and verifiable. A single-file patch is easy for a human reviewer to verify in the PR diff. Cross-file changes can be done by the developer after reviewing the AI PR.

Trade-off: Cannot fix issues that span multiple files (e.g., extracting a shared constant to a new utility class).

Why self-hosted GraphiQL assets instead of CDN?

Rationale: The default Spring Boot GraphiQL loads React, ReactDOM, and GraphiQL JS from unpkg.com CDN. In air-gapped or enterprise environments with strict CSP policies, CDN loads fail silently resulting in a blank UI. Bundling the assets locally guarantees consistent behaviour regardless of network policy.

Trade-off: Adds ~3 MB to the Docker image. GraphiQL version must be manually updated when upgrading.

Why Gradle Groovy DSL instead of Kotlin DSL?

Rationale: Groovy DSL has better tooling compatibility with legacy IDE versions and cleaner syntax for Spring Boot convention plugins. The project targets Java 17 developers who may be unfamiliar with Kotlin DSL nuances. Groovy DSL compiles faster on first run.

Trade-off: Groovy DSL is less type-safe than Kotlin DSL — typos in build.gradle may only surface at runtime.

Document End

Multi-Agent GitHub Code Review & Auto-Fix System — Architecture Document v1.0.0